

BỘ CHUẨN BỊ PHẢN BIỆN — ĐATN

"Hệ thống liên camera AI tracking đối tượng và dự đoán hướng đi"

Tài liệu tổng hợp: model/thuật toán · nguyên lý · lý do chọn model + tham số · so sánh với kết quả trước · vị trí code · bộ câu hỏi phản biện. Bám theo DoAn.pdf (bản đóng băng) + code thực tế.

0. Tóm tắt hệ thống (đọc đầu tiên)

Hệ thống giám sát giao thông chạy trên **video CCTV thật** ở đô thị Việt Nam (mật độ xe máy cao). Thay vì một mô hình end-to-end, hệ thống là **pipeline AI ghép nối nhiều mô hình chuyên biệt**. Ứng dụng web **client-server** (backend FastAPI/Python, frontend React trên trình duyệt), pipeline AI trên PyTorch, cần **GPU NVIDIA phổ thông** để chạy thời gian thực (~15–20 FPS). Đóng góp chính là **Goal Classifier** — dự đoán hướng rẽ của phương tiện trong bối cảnh giao thông VN.

Bốn nguyên tắc thiết kế xuyên suốt (dùng để trả lời mọi câu "tại sao chọn cách này"): 1. **Nhẹ & nhanh** — real-time nhiều camera trên GPU phổ thông (YOLO11, ByteTrack). 2. **Ít phụ thuộc dữ liệu lớn** — hợp lượng dữ liệu thật ít (Gradient Boosting, Kalman, rule-based). 3. **Minh bạch, diễn giải được** — hiệu chỉnh xác suất, hệ luật thay cho học máy hộp đen. 4. **Dữ liệu/công cụ mở** — áp thẳng lên hạ tầng CCTV thật (OSM, SUMO, FastAPI, React, Leaflet).

1. Bảng tổng hợp pipeline

Luồng: **Đọc frame** → **Phát hiện** → **Theo dõi** → **Quỹ đạo ngắn hạn** → **Hướng rẽ** → **Vi phạm** → **Bản đồ/hành trình** → **Hiển thị**.

#	Thành phần	Model / Thuật toán	Code	Số liệu chính
1	Phát hiện đối tượng	YOLO11s fine-tune VN	<code>custom_tracking_system/modules/detector.py</code> (<code>yolo11s_vn.pt</code>)	mAP50 89.5% , mAP50-95 77.0%
2	Theo dõi 1 camera	ByteTrack (boxmot)	<code>custom_tracking_system/modules/tracker.py</code>	track_thresh 0.25 / match 0.8 / buffer 30
3	Quỹ đạo ngắn hạn	Kalman Filter (gia tốc không đổi)	<code>custom_tracking_system/modules/trajectory_predictor.py</code>	dự đoán vài giây, không cần train
4	Hướng rẽ (đóng góp chính)	Gradient Boosting + Platt , 33 đặc trưng	<code>custom_tracking_system/modules/goal_classifier.py</code> (<code>goal_classifier_real.pkl</code>)	acc 60.12% , balanced 54.18% , ECE 0.101
5	Vi phạm	Hệ luật (rule-based) + đèn HSV	<code>incident_detector.py</code> , <code>traffic_light_classifier.py</code> , <code>alert_system.py</code>	đèn đỏ/vạch dừng/sai làn
6	Bản đồ/hành trình (bổ trợ, định tính)	Khớp hình học trên junction graph (OSM→SUMO→OpenDRIVE)	<code>custom_tracking_system/route_predictor.py</code> , Map/	101 nút / 742 đoạn
7	Serving	FastAPI (REST+WS+MJPEG) + React/Leaflet + SQLite	<code>server/app.py</code> , <code>server/services/ai_processor.py</code>	—

2. Chi tiết từng thành phần

2.1 Phát hiện đối tượng — YOLO11s_vn

Nguyên lý: mô hình một giai đoạn, anchor-free. Backbone (CSP + SPPF) trích đặc trưng → Neck (FPN + PAN) tổng hợp đa tỉ lệ → Head dự đoán trực tiếp box + lớp + độ tin cậy. Hậu xử lý: lọc theo ngưỡng tin cậy + **NMS** (khử box trùng theo IoU). 5 lớp: person/car/motorcycle/bus/truck.

Tại sao chọn YOLO11s: - Real-time nhiều camera trên GPU phổ thông → **loại two-stage** (Mask R-CNN chậm vài FPS). - Anchor-free + cải thiện small-object so v5/v8 → hợp **xe máy nhỏ/xạ** trong ảnh CCTV. - Hệ sinh thái Ultralytics có sẵn công cụ **fine-tune** trên nhãn tùy chỉnh.

Tại sao fine-tune (không dùng COCO gốc): COCO 80 lớp generic, không hợp mật độ + góc nhìn từ trên cao của CCTV VN; gộp về 5 lớp giao thông giúp học đúng bối cảnh, tăng recall xe máy.

Tham số & lý do: - `conf = 0.35` — hạ so mặc định, **đọc từ PR curve**; lý do: trong giám sát **bỏ sót một xe nguy hại hơn phát hiện dư** (dư dễ bị bước theo dõi lọc sau) → ưu tiên recall. - `imgsz = 640` — cân bằng tốc độ/độ chính xác khi chạy song song nhiều camera. - **FP16 + batch detection** (gom nhiều camera 1 lượt suy luận) — tận dụng GPU.

So sánh với trước: (xem chi tiết Mục 3.1) per-class mAP50: person 89.9%, car 98.9%, motorcycle 98.4%, bus 95.3%, truck 89.2%. **Không có số stock-COCO đo trên tập VN** — dùng lập luận "stock không làm được task 5 lớp VN" (Mục 3.1).

Code: `custom_tracking_system/modules/detector.py` (lớp `ObjectDetector`) · gọi ở `server/services/ai_processor.py:467` (`detect_batch`).

2.2 Theo dõi trong một camera — ByteTrack

Nguyên lý: tracking-by-detection. Mỗi frame: **Kalman Filter** dự đoán vị trí kế tiếp của từng track → ghép cặp box mới với track cũ bằng **thuật toán Hungarian**. Điểm riêng của ByteTrack (chiến lược **BYTE**): giữ và ghép **cả box tin cậy thấp** (xe bị che khuất một phần) để không đứt track; chỉ không dùng box tin cậy thấp để khởi tạo track mới. Mỗi xe được gán **Track ID** ổn định trong phạm vi một camera.

Tại sao chọn ByteTrack (không SORT / DeepSORT): - **vs SORT:** SORT loại box tin cậy thấp → xe máy đi sát nhau, che khuất liên tục sẽ **đứt track, sinh ID mới** không cần thiết. BYTE giữ được track. - **vs DeepSORT/StrongSORT:** hai loại này thêm mạng trích đặc trưng hình ảnh mỗi frame → **tốn tính toán, dư thừa** khi chỉ cần định danh trong một camera.

Tham số & lý do: `track_thresh=0.25` (ngưỡng kích hoạt track), `match_thresh=0.8` (ngưỡng ghép cặp), `track_buffer=30 frame` (giữ track mất dấu trước khi xoá) — hợp giao thông che khuất tạm thời; `frame_rate` đặt riêng theo camera.

So sánh: MOTA/IDF1 được ghi nhận riêng ở báo cáo (Mục 5.1). Lưu ý trung thực: **MOTA bị thổi phồng cấu trúc** (FN/FP≈0 do cách thu GT), chỉ **IDF1 có ý nghĩa**.

Code: `custom_tracking_system/modules/tracker.py` (lớp `ByteTrackWrapper`) · gọi ở `ai_processor.py:477`.

2.3 Dự đoán quỹ đạo ngắn hạn — Kalman Filter

Nguyên lý: bộ lọc Kalman mô hình **gia tốc không đổi**, vector trạng thái 6 chiều `[x, y, vx, vy, ax, ay]`. Luân phiên **predict** (chiếu trạng thái tới trước vài giây) và **update** (hiệu chỉnh bằng đo đạc mới).

Tại sao Kalman (không mạng học sâu quỹ đạo): không phụ thuộc dữ liệu huấn luyện → chạy ngay trên camera/khu vực mới; đủ chính xác cho dự đoán ngắn hạn (vài giây). Dự đoán dài hạn (qua nhiều ngã tư) do cơ chế trên đồ thị đường đảm nhiệm, không phải Kalman.

Tham số: `process_noise_std=50`, `measurement_noise_std=5`, window 10.

Code: `custom_tracking_system/modules/trajectory_predictor.py` (`KalmanFilter2D`, `TrajectoryPredictor`) · gọi ở `ai_processor.py:506`.

Lưu ý trình bày: **báo cáo dùng Kalman**. Code có kèm ensemble Kalman+GRU nhưng **không trình bày GRU** (xem Mục 6).

2.4 Phân loại hướng rẽ — Goal Classifier (ĐÓNG GÓP CHÍNHH)

Model: `Gradient Boosting Classifier` + hiệu chỉnh xác suất **Platt scaling** (`CalibratedClassifierCV`), scikit-learn, **33 đặc trưng thủ công**. 4 lớp: `straight / left / right / u_turn`. File: `goal_classifier_real.pkl` + scaler + label encoder + feature_names.

Nguyên lý hoạt động (từng bước): 1. Giữ lịch sử quỹ đạo mỗi track (deque, cửa sổ 4s), lưu tâm bbox + kích thước. 2. Chưa đủ 10 frame (~1.5s) → chưa dự đoán. 3. Trích **33 đặc trưng** động học trên horizon **1.5s** gần nhất, 5 nhóm: - **Vận tốc** (sai phân hữu hạn): mean/std/last/recent/change/change_rate. - **Chuẩn hoá theo phối cảnh:** `speed_norm_bbox` (theo cỡ bbox), `speed_ms_*` (đổi ~m/s qua `px_per_m = cỡ_bbox / kích_thước_thật_theo_lớp`), `speed_norm` (theo ngưỡng lớp) — để so sánh xe gần/xa công bằng. - **Hướng:** `head_sin/cos`, `head_rate`, `total_abs_heading` (rẽ nhiều/ít), `total_sign_heading` (dấu → trái/phải), `heading_range`, `head_monotonic` (rẽ đơn điệu?). - **Giảm tốc:** `speed_min`, `speed_drop`, `speed_min_ratio`, `speed_final_third`. - **Gia tốc ngang** (lateral accel `|vx·ay - vy·ax|/|v|`) + one-hot loại xe. 4. **Heuristic chặn nhiễu:** nếu `total_abs_heading < 12°` (đi gần thẳng) → ép `straight` xác suất 1.0, bỏ qua model. 5. Chuẩn hoá (StandardScaler) → `predict_proba` → 4 xác suất; **Platt scaling** cho xác suất trung thực (đo bằng ECE). 6. Trả về: hướng (argmax), 4 xác suất, độ tin cậy (top-1).

Tại sao Gradient Boosting (không LSTM/GRU/Transformer): dữ liệu thật chỉ **vài nghìn track** — đủ cho cây tăng cường trên đặc trưng bám vật lý, **chưa đủ cho học sâu học biểu diễn từ đầu mà không quá khớp**. Ngoài ra: minh bạch + cho **xác suất hiệu chỉnh** (yêu cầu hiển thị độ tin cậy không phóng đại).

Tham số & lý do: - `horizon = 1.5s` — chọn từ **khảo sát horizon** (0.5/1.0/1.5/2.0s); ngắn quá thiếu thông tin, dài quá hành vi bất định; tốt nhất vùng 1.0–1.5s. - **Class weights** `straight=1.0, left=2.5, right=2.5, u_turn=6.0` — bù mất cân bằng (straight áp đảo), ép model học các lớp rẽ thiếu số. - `junction_frac=0.65` — tỉ lệ mẫu ở vùng giao lộ. - **Platt scaling** — hạ ECE, xác suất phản ánh đúng tần suất thật.

Số liệu: acc **60.12%**, balanced acc **54.18%**, ECE **0.101** (5.332 train / 1.334 test, horizon 1.5s). Yếu nhất ở `u_turn` do ít mẫu.

So sánh (RẤT QUAN TRỌNG — xem Mục 3.2): đây là chỗ dễ bị hỏi "sao chỉ 60%". Câu trả lời: raw accuracy đánh lừa (đoạn 'straight' luôn đã ~68%); đóng góp là **balanced accuracy + phát hiện lớp rẽ + calibration**.

Code: `custom_tracking_system/modules/goal_classifier.py` (`GoalClassifier.update_and_predict / _extract_features`) · gọi ở `ai_processor.py:550` . Script train: `custom_tracking_system/scripts/train/train_goal_classifier_real.py` .

2.5 Phát hiện vi phạm — Hệ luật (rule-based)

Nguyên lý: không học máy, dùng **luật hình học/động học**. - **Trạng thái đèn:** phân tích màu trong không gian **HSV** tại toạ độ bóng đèn do người vận hành khai báo + bỏ phiếu đa số chống nhiễu; dự phòng theo chu kỳ đèn. - **Vượt đèn đỏ / vạch dừng:** đèn đỏ + xe **đang di chuyển** (speed > ngưỡng) vượt vạch vào giao lộ. - **Sai làn:** mỗi làn là polygon kèm *lớp xe được phép* + *hướng hợp lệ*; xác định làn theo tâm box, so hướng dịch chuyển với hướng hợp lệ bằng **tích vô hướng (dot product)**. - Chống nhiễu: yêu cầu vi phạm ổn định qua nhiều frame + **cooldown** (~4s) giữa hai cảnh báo cùng loại/cùng xe.

Tại sao rule-based (không unsupervised anomaly detection): minh bạch/diễn giải được ("vượt vạch khi đèn đỏ") và dễ chỉnh ngưỡng theo từng camera **không cần train lại**; unsupervised đòi hỏi nhiều dữ liệu "bình thường" đặc thù và chỉ báo "khác thường" khó giải thích. Vi phạm thật hiếm, khó thu đủ để kiểm chứng.

Code: `custom_tracking_system/modules/incident_detector.py` (`IncidentDetector`), `traffic_light_classifier.py` , `alert_system.py` .

Trên video Phố Huế demo: **chỉ bật "sai làn"**; đèn đỏ/vạch dừng cần cấu hình ROI + trạng thái đèn tương ứng.

2.6 Bản đồ / suy diễn hành trình — hỗ trợ, định tính

Nguyên lý: khớp hướng rẽ dự đoán (2.4) với hình học các đoạn đường ra khỏi giao lộ trên **đồ thị đường** dựng từ OpenStreetMap → SUMO → **OpenDRIVE** (101 nút / 742 đoạn), lập nhiều hop → suy ra hành trình khả dĩ; quy pixel→thế giới bằng ground-plane projection, hiển thị Leaflet.

Định vị khi trình bày: đây là **tính năng trực quan hoá phụ trợ, mang tính định tính** (calibration bằng mắt, sai số vài mét — báo cáo tự nhận), **không phải đóng góp chính**. Đóng góp có số liệu là Goal Classifier (2.4).

Code: `custom_tracking_system/route_predictor.py` , `Map/` (junction graph + georeference).

2.7 Tầng phục vụ (serving)

FastAPI (async): REST API (cấu hình/lịch sử) + **WebSocket** (đẩy cảnh báo/thống kê) + **MJPEG** (luồng ảnh đã annotate). Frontend **React** (Vite + Tailwind) + **Leaflet/OSM**; lưu **SQLite**. Pipeline AI chạy background thread, chia sẻ trạng thái; đáp ứng nhiều camera bằng **batch detection** + điều tiết tần suất các bước nặng.

Code: `server/app.py` , `server/routers/` , `server/services/ai_processor.py` .

3. So sánh & Ablation (số thật để bảo vệ)

3.1 YOLO — so với "trước khi fine-tune"

⚠️ **Không có số stock-COCO đo trên tập test VN** (chưa ai đo, không có trong báo cáo, không chạy lại được). **Đừng bịa con số.**

Số CÓ SẴN (model đã fine-tune, 800 ảnh auto_label):

Lớp	mAP50
person	89.9%
car	98.9%
motorcycle	98.4%
bus	95.3%
truck	89.2%
Tổng	89.5% (mAP50-95 = 77.0%)

Tham chiếu "trước fine-tune": YOLO11s gốc (pretrained COCO) — chính thức Ultralytics **mAP50-95 = 47.0% trên COCO** (khác tập dữ liệu, phải nói rõ).

Câu trả lời chuẩn khi bị hỏi "so với trước fine-tune?":

"Không đo được stock trực tiếp trên tập VN vì model gốc xuất 80 lớp COCO, chỉ số lớp khác 5 lớp VN → đối chiếu cho mAP≈0, vô nghĩa. Quan trọng hơn: 'motorcycle' của COCO là mô-tô kiểu phương Tây chụp ngang, còn bài toán là xe máy dày đặc nhìn từ CCTV trên cao. Fine-tune **không phải để tăng vài %, mà là điều kiện bắt buộc** để model chạy đúng bộ lớp + domain. Minh chứng model học đúng: per-class mAP50 89–99% + confusion matrix chỉ nhầm giữa các lớp hình dáng tương tự."

3.2 Goal Classifier — so với phiên bản trước (số thật)

Phiên bản	Đặc trưng	Accuracy thô	Chất lượng phát hiện rõ
v0 baseline (h=1.0, không class-weight)	26	68.3%	f1_macro 0.49 — straight 99.9% nhưng left 31% / right 19% / u_turn 25% (≈ đoán "thẳng" mọi xe)
Bản production (h=1.5, class-weight + Platt)	33	60.1%	balanced acc 54.2% , ECE 0.10 — phát hiện rõ tốt hơn hẳn

⚠️ **Điểm mấu chốt:** accuracy thô **GIẢM 68%→60% nhưng model TỐT LÊN**. Vì bài toán 4 lớp mất cân bằng — chỉ cần đoán 'straight' luôn là đã ~68% nhưng **vô dụng** (không phát hiện rõ). Ta tối ưu **balanced accuracy + calibration** để thực sự phát hiện left/right/u_turn.

Câu trả lời chuẩn khi bị hỏi "sao chỉ 60%?":

"Em cố tình không tối ưu accuracy thô. Trong bài 4 lớp mất cân bằng, đoán 'đi thẳng' cho mọi xe đã ~68% accuracy nhưng vô dụng — không phát hiện được rõ. Em tối ưu **balanced accuracy** (54.2% so với random 25%) và **hiệu chỉnh xác suất** (ECE=0.10) để mô hình thật sự phân biệt hướng rõ và cho độ tin cậy trung thực."

Ablation có thể nêu thêm: 26→33 đặc trưng (thêm nhóm chuẩn hoá bbox) · class-weight none→custom · có/không Platt (accuracy ~giữ, ECE cải thiện) · horizon sweep 0.5/1.0/1.5/2.0.

3.3 So sánh CARLA vs dữ liệu thật (Goal Classifier)

CARLA baseline ~**70.3% acc**, dữ liệu thật **60.12%**. **Câu trả lời nếu bị hỏi "sao thật thấp hơn mô phỏng?":**

"Dữ liệu thật khó hơn hẳn: nhãn tự động (auto-label) có nhiều, giao thông hỗn hợp xe máy dày đặc, che khuất, hành vi đa dạng hơn môi trường mô phỏng CARLA sạch. Con số thật phản ánh đúng độ khó thực tế."

4. Bộ câu hỏi phản biện & câu trả lời mẫu

Q: Tại sao dùng pipeline nhiều model thay vì một mô hình end-to-end?

Tận dụng các kiến trúc đã kiểm chứng, cho phép tinh chỉnh từng thành phần trên dữ liệu VN, minh bạch/dễ debug, và hợp lượng dữ liệu thật ít. End-to-end đòi hỏi dữ liệu gán nhãn khổng lồ mà đề tài không có.

Q: Tại sao YOLO11s mà không phải model chính xác hơn (two-stage)? → Mục 2.1 (real-time nhiều camera).

Q: $conf=0.35$ sao thấp thế? → Từ PR curve; bỏ sót nguy hại hơn phát hiện dư trong giám sát → ưu tiên recall.

Q: Tại sao Gradient Boosting mà không dùng deep learning (LSTM)? → Dữ liệu vài nghìn track, không đủ cho deep learning; cây tăng cường + đặc trưng vật lý + calibration phù hợp hơn.

Q: Goal Classifier chỉ 60% có thấp không? → Mục 3.2 (balanced accuracy, không phải raw accuracy).

Q: So với trước khi fine-tune YOLO tốt hơn bao nhiêu? → Mục 3.1 (không có số stock, dùng lập luận bộ lớp).

Q: Tại sao ByteTrack không DeepSORT? → Mục 2.2 (xe máy che khuất + nhẹ).

Q: Tại sao dùng hệ luật cho vi phạm, không dùng AI? → Minh bạch, ít phụ thuộc dữ liệu, dễ chỉnh ngưỡng (Mục 2.5).

Q: Bản đồ/route dự đoán có đáng tin không? → Là **trực quan hoá định tính** dựa trên hướng rẽ + hình học đường; calibration bằng mắt (sai số vài mét), không phải đóng góp có số liệu. Đóng góp định lượng là Goal Classifier.

Q: Nhận dạng biển số hoạt động chưa? → Đã train 2 giai đoạn nhưng **chưa tích hợp vào pipeline**; xe máy gần như không đọc được biển ở khoảng cách CCTV → để **Chương 6 (future work)**.

Q: Chạy được bao nhiêu FPS / máy camera? → Mục tiêu 6 camera @15–20 FPS trên GPU phổ thông; trên máy CPU chỉ ~0.5–2 FPS (bottleneck YOLO ~490ms/frame trên CPU). FPS 6-camera **chưa đo chính thức**.

5. Điểm yếu & cách phòng thủ (biết trước để không bị bất ngờ)

Điểm yếu	Cách phòng thủ (trả lời trung thực)
Goal Classifier acc chỉ 60%	Dùng khung balanced accuracy + calibration ; raw accuracy đánh lừa (Mục 3.2)
Không có baseline stock YOLO trên tập VN	Lập luận "bộ lớp không khớp, stock không làm được task" (Mục 3.1); đừng bịa số
Nhãn train/test là auto-label (chưa hand-verify)	Khai rõ là giới hạn; đây là đánh đổi để có đủ dữ liệu thật VN
Biển số chưa hoạt động	Để Chương 6 future work; đã nêu trong báo cáo
ReID xe máy phân biệt kém (~0.60)	Re-ID là hạ tầng phụ trợ , không phải đóng góp chính (báo cáo đã reframe)
FPS 6-camera chưa đo	Mục tiêu thiết kế; đã có batch detection + throttle, cần máy GPU để đo
Route/map định tính	Định vị là visualization, không phải đóng góp định lượng
Real < CARLA (60% < 70%)	Dữ liệu thật khó hơn, nhãn nhiều, giao thông hỗn hợp (Mục 3.3)

6. Những thứ CÓ trong code nhưng KHÔNG trình bày (để nhất quán)

Để lời nói khớp báo cáo, **KHÔNG** nêu các thứ sau khi bảo vệ (chúng còn trong code nhưng không thuộc câu chuyện báo cáo):

- **ReID xuyên camera / OSNet / global_tracking** — chỉ là hạ tầng phụ; không nằm trong 4 thành phần chính. (Trong demo 1 camera đã throttle mạnh để giảm tải.)
- **GRU trajectory (ensemble Kalman+GRU)** — báo cáo chỉ trình bày **Kalman**.
- **CARLA** — báo cáo **chỉ dùng dữ liệu/demo thật** (Phố Huế), loại CARLA hoàn toàn.

Route predictor thì **giữ** (báo cáo có mô tả Mục 3.5.2/UC3.3), nhưng định vị là **bổ trợ định tính** — không phủ nhận, không nhấn mạnh.